МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

# ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ» за 5 семестр

Тема: «Введение в нейронные сети:
построение многослойного перцептрона»

Выполнил:
студент 3 курса
группы АС-67
Ковалевич М. М.

Проверил:
Крощенко А. А.

2026

**Цель работы:** построить, обучить и оценить **многослойный перцептрон (MLP)** для решения задачи классификации

**Общее задание:**

1. **Импорт библиотек и подготовка данных**
   - импортируйте `torch, torch.nn, torch.optim`, а также `sklearn` для загрузки данных и их предобработки;
   - загрузите датасет, выполните стандартизацию (`StandardScaler`) и кодирование признаков;
   - разделите данные на обучающую и тестовую выборки;
   - **преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32).`**
2. **Определение архитектуры нейронной сети**
   - создайте класс, наследуемый от `torch.nn.Module`;
   - в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear, nn.ReLU, nn.Dropout`);
   - в методе `forward` опишите последовательность применения слоев к входным данным.
3. **Инициализация модели, функции потерь и оптимизатора**
   - создайте экземпляр вашей модели: `model = MLP();`
   - определите функцию потерь. Для бинарной классификации
   - используйте `nn.BCEWithLogitsLoss`, для многоклассовой –
   - `nn.CrossEntropyLoss`;
   - определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss;`
4. **Написание цикла обучения (Training Loop)**
   - запустите цикл на определенное количество эпох;
   - внутри цикла:
     1) переведите модель в режим обучения: `model.train();`
     2) сделайте предсказание (forward pass): `y_pred = model(X_train);`
     3) рассчитайте потери (loss): `loss = criterion(y_pred, y_train);`
     4) обнулите градиенты: `optimizer.zero_grad();`
     5) выполните обратное распространение ошибки: `loss.backward();`
     6) сделайте шаг оптимизации: `optimizer.step().`
5. **Оценка модели (Evaluation)**

   - переведите модель в режим оценки: `model.eval();`

- используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
- сделайте предсказания на тестовых данных;
- преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога > 0);
- рассчитайте метрики (`accuracy, f1-score` и т.д.), используя `sklearn.metrics`.

**Задания по вариантам:**

**Вариант 5 Определение съедобности грибов**

• Mushroom Classification

• **Задача:** определить, является ли гриб ядовитым или съедобным(бинарная классификация)

• **Архитектура:**

- Входной слой;
- Один скрытый слой с 8 нейронами (ReLU)
- Выходной слой с 1 нейроном(Sigmoid)

• **Эксперимент:** попробуйте использовать tanh в качестве функции активации в скрытом слое вместо ReLU. Сравните итоговую точность.

**Код программы:**

```
import torch
from torch import nn
from torch import optim

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, f1_score

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Платформа обучения: ", device)


class MLP(nn.Module):
    def __init__(self, in_features: int, activation: str = 'relu') -> None:
        super().__init__()
        act = nn.ReLU() if activation == 'relu' else nn.Tanh()
        self.linear_stack = nn.Sequential(
```

```
            nn.Linear(in_features=in_features, out_features=8),  # скрытый слой,
8 нейронов
            act,                                                 # ReLU или Tanh
            nn.Linear(in_features=8, out_features=1),            # выход: 1
нейрон (логит)
        )

    def forward(self, x):
        return self.linear_stack(x)


def train_and_evaluate(activation: str, data, epochs: int = 200) -> float:
    X_train_t, X_test_t, y_train_t, y_test_t = data

    torch.manual_seed(42)  # одинаковая инициализация для сравнения
    model = MLP(in_features=X_train_t.shape[1], activation=activation).to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.01)

    print("=" * 45)
    print(f"\nАктивация скрытого слоя: {activation}")
    print("\nОбучаемая модель: \n", model)
    print("\tКоличество эпох: ", epochs)
    print("\tОптимизатор(Adam): ")
    for name, val in optimizer.defaults.items():
        print("\t\t", name, " : ", val)
    print("\tФункция потерь(бинарная): ", criterion)
    print("=" * 45)

    # Цикл обучения
    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 50 == 0:
            print(f"Эпоха {epoch + 1}/{epochs}, loss = {loss.item():.4f}")

    print("\nОценка модели нейронной сети: ")
    model.eval()
    with torch.no_grad():
        logits = model(X_test_t)
        predicted_classes = (logits > 0).float()

    y_true = y_test_t.cpu().numpy().ravel()
    y_hat = predicted_classes.cpu().numpy().ravel()
```

```
    print(classification_report(y_true, y_hat, target_names=['съедобный (e)',
'ядовитый (p)']))
    print("f1-score =", f1_score(y_true, y_hat))
    return accuracy_score(y_true, y_hat)


def solution() -> None:
    # 1. загрузка данных и кодирование признаков
    df = pd.read_csv('mushrooms.csv', sep=',')
    y = df['class'].map({'e': 0, 'p': 1})  # ядовитый = 1
    X = pd.get_dummies(df.drop(columns='class'), drop_first=True).astype(float)

    # 2. разделение выборки
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, train_size=0.7, random_state=42, stratify=y)

    # 3. стандартизация
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # 4. тензоры
    X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
    X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
    y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1,
1).to(device)
    y_test_t = torch.tensor(y_test.values, dtype=torch.float32).view(-1,
1).to(device)
    data = (X_train_t, X_test_t, y_train_t, y_test_t)

    # 5. эксперимент: ReLU vs Tanh
    results = {}
    for activation in ['relu', 'tanh']:
        results[activation] = train_and_evaluate(activation, data)

    print("\n" + "=" * 45)
    print("Сравнение итоговой точности (accuracy):")
    for activation, acc in results.items():
        print(f"\t{activation:5s}: {acc:.4f}")


if __name__ == "__main__":
    solution()
```

```
Платформа обучения:  cpu
==========================================

Активация скрытого слоя: relu

Обучаемая модель:
 MLP(
  (linear_stack): Sequential(
    (0): Linear(in_features=95, out_features=8, bias=True)
    (1): ReLU()
    (2): Linear(in_features=8, out_features=1, bias=True)
  )
)
        Количество эпох:  200
        Оптимизатор(Adam):
                lr  :  0.01
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(бинарная):  BCEWithLogitsLoss()
==========================================
Эпоха 50/200, loss = 0.0090
Эпоха 100/200, loss = 0.0037
Эпоха 150/200, loss = 0.0029
Эпоха 200/200, loss = 0.0026

Оценка модели нейронной сети:
              precision    recall  f1-score   support

съедобный (e)       1.00      1.00      1.00      1263
 ядовитый (p)       1.00      1.00      1.00      1175

     accuracy                           1.00      2438
    macro avg       1.00      1.00      1.00      2438
 weighted avg       1.00      1.00      1.00      2438

f1-score = 0.9974489795918368
==========================================
```

Рис. 1. Результат программы с обучением нейронной сети с одним скрытым слоем на 8 нейронов (ReLU, 200 эпох)

```
==========================================
Активация скрытого слоя: tanh

Обучаемая модель:
 MLP(
  (linear_stack): Sequential(
    (0): Linear(in_features=95, out_features=8, bias=True)
    (1): Tanh()
    (2): Linear(in_features=8, out_features=1, bias=True)
  )
)
        Количество эпох:  200
        Оптимизатор(Adam):
                lr  :  0.01
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(бинарная):  BCEWithLogitsLoss()
==========================================
Эпоха 50/200, loss = 0.0212
Эпоха 100/200, loss = 0.0080
Эпоха 150/200, loss = 0.0047
Эпоха 200/200, loss = 0.0031

Оценка модели нейронной сети:
               precision    recall  f1-score   support

съедобный (e)       1.00      1.00      1.00      1263
 ядовитый (p)       1.00      1.00      1.00      1175

     accuracy                           1.00      2438
    macro avg       1.00      1.00      1.00      2438
 weighted avg       1.00      1.00      1.00      2438

f1-score = 0.9991482112436116

==========================================
```

Рис. 2. Результат программы с обучением нейронной сети с одним скрытым слоем на 8 нейронов (tanh, 200 эпох)

Таблица 1. Оценки нейронных сетей

| Активация | Accuracy | F1-score | Loss на 200-й эпохе |
|---|---|---|---|
| ReLU | 0.9975 | 0.9974 | 0.0026 |
| tanh | 0.9992 | 0.9991 | 0.0031 |

**Вывод:** в ходе лабораторной работы была обучена нейронная сеть с одним скрытым слоем на 8 нейронов для классификации грибов. Точность на тестовой выборке с активацией ReLU составила 0.9975, с tanh – 0.9992. Обе модели показали практически идеальное качество, разница между ними незначительна.